主要有如下题型：**概念题、语法设计、RE->NFA->DFA->miniDFA、消除 xxx、LL（1）表构建、LR（1）表构建、根据 SDD 构建带注释的语法树、C 代码转三地址码、基本块转 DAG、C 代码推过程调用时的控制栈内存分配**。

（可能）基于构建的 LL（1）表分析、消除伊普西隆、非二义性文法、LR（0）、LALR、SDT、三地址码变成三元组或者四元组的形式。

概念题

概念：第一章的那些内容（比如编译器各个部件的 role 和 task、编译器解释器混合编译器优缺点）、四类语法定义（主要是文法）

文法 (grammar)

表达语言构成规则的形式化方法

$G=(V_N, V_T, S, P)$

V_N:非终结符集

V_T:终结符集

S:文法开始符号

P:产生式　　$A\rightarrow\alpha$

1.语法设计

感觉主要有根据算式结构直接给出语法和画状态转移图两类方法。

例：设$L_2=\{a^i b^j c^k \mid i, j, k>=1$ 且$a, b, c\in V_T\}$，试构造生成L_2的文法G_2。

1) 根据算式结构直接给出语法

(0) $S \rightarrow ABC$

(1) $A \rightarrow aA \mid a$

(2) $B \rightarrow bB \mid b$

(3) $C \rightarrow cC \mid c$

2) 画状态转移图（状态为当前已经读到的字符串决定）

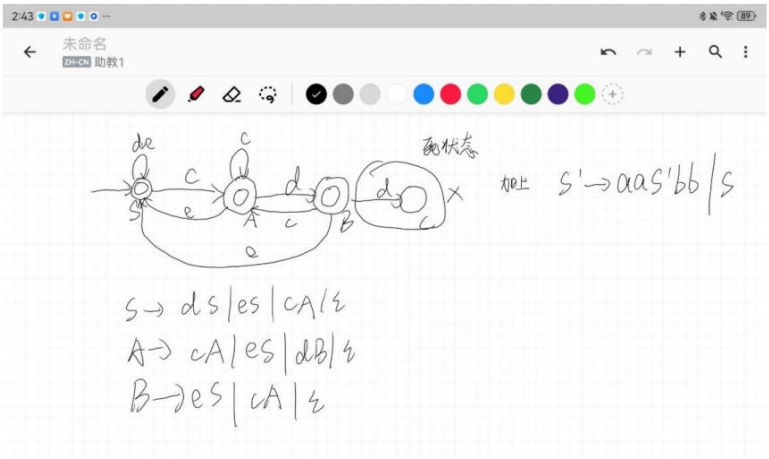
(0) $S \rightarrow aS \mid aB$

(1) $B \rightarrow bB \mid bC$

(2) $C \rightarrow cC \mid c$

下面这题综合了两个方法的精髓。

6. $\{a^{2m}\omega b^{2m} \mid m\geq 0, \omega\in\{c,d,e\}^*$ and ω does not contain the substring $cdd\}$.

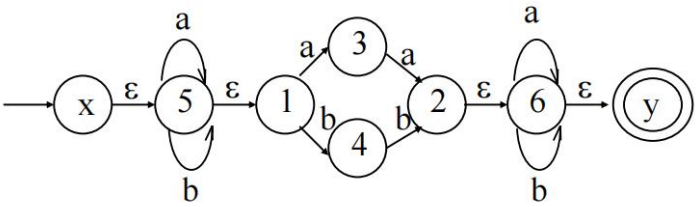


2）可以再看看奇数偶数和余数为几那种题。

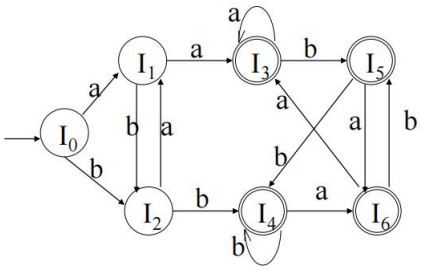
2. RE->NFA->DFA->miniDFA

以一道例题为例讲解如下

(a|b)*(aa|bb)(a|b)*

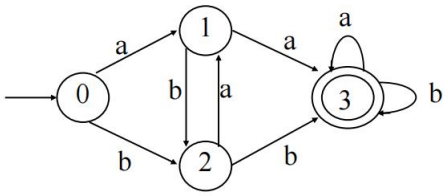


I	a	b
I_0={x,5,1}	I_1={5,3,1}	I_2={5,4,1}
I_1={5,3,1}	I_3={5,3,2,1,6,y}	I_2={5,4,1}
I_2={5,4,1}	I_1={5,3,1}	I_4={5,4,1,2,6,y}
I_3={5,3,2,1,6,y}	I_3={5,3,2,1,6,y}	I_5={5,1,4,6,y}
I_4={5,4,2,1,6,y}	I_6={5,3,1,6,y}	I_4={5,4,1,2,6,y}
I_5={5,1,4,6,y}	I_6={5,3,1,6,y}	I_4={5,4,1,2,6,y}
I_6={5,3,1,6,y}	I_3={5,3,2,1,6,y}	I_4={5,1,4,6,y}



Π_0 = { {0,1,2},{3,4,5,6} }

Π_1 = { {1}, {0}, {2}, {3,4,5,6} }



算法3.23 将正则表达式转换为一个NFA的McMaughton-Yamada-Thompson算法。

书 3.7.4

依据几种归纳规则来推算（连接、|、*）

算法3.20 由NFA构造DFA的子集构造（subset construction）算法。

操作	描述
ϵ-$closure(s)$	能够从NFA的状态 s 开始只通过 ϵ 转换到达的 NFA状态集合
ϵ-$closure(T)$	能够从 T 中某个NFA状态 s 开始只通过 ϵ 转换到达的NFA状态集合，即 $\cup_{s \in T} \epsilon$-$closure(s)$
$move(T, a)$	能够从 T 中某个状态 s 出发通过标号为 a 的转换到达的NFA状态的集合

图3-31 NFA状态集上的操作

书 3.7.1

初始为 s0 状态的伊普西隆闭包，以此初始化队列。进入循环，取一个状态闭包然后检验所有可能的输入符号，算出闭包，得出状态闭包（若为新则加入队列）。每次取过的状态闭包不再重复取。

算法3.39 最小化一个DFA的状态数量。

书 3.9.6

不断划分组，最终每个组都等价，再各组取一个作为代表构建 miniDFA。

3.消除 xxx

消除伊普西隆

无伊普西隆产生式的上下文无关文法要满足条件

- 若文法 P 中含 S → 伊普西隆 ，则 S 不出现在任何产生式右部，其中 S 为文法的开始符号；
- 文法 P 中不再含有其它任何伊普西隆产生式

The algorithm to construct a context-free grammar without ε-production： $G=(V_N,V_T,P,S)$ ⟹ $G'=(V'_N,V'_T,P',S')$

(1) Find out all non-terminal symbols that can derive ε after some steps, and put them into the set V_0;

(2)Construct the P' set of productions of G' as following steps:

(A)If an symbol in V_0 appears in the right-side of a production, change the production into two productions：substitute the symbol in ε and itself in the production respectively； put the new productions into P'

(B)Otherwise, put the productions relating to the symbol into P' except for ε-production relating to the symbol

(C)If there exists the production of the form $S\rightarrow \varepsilon$ in P, change the production into $S'\rightarrow \varepsilon \mid S$ and put them into P',let S' be the Start Symbol of G'， let $V'_N=V_N\cup\{S'\}$.

设$G_1=(\{S\},\{a,b\},P,S)$,其中

P: (0) $S\rightarrow\varepsilon$ (1) $S\rightarrow aSbS$ (2) $S\rightarrow bSaS$

答案:

(1) $V_0=\{S\}$

(2) P': $S\rightarrow abS \mid aSbS \mid aSb \mid ab$

$S\rightarrow baS \mid bSaS \mid bSa \mid ba$

$S'\rightarrow \varepsilon \mid S$

(3) $G_1'=(\{S',S\},\{a,b\},P',S')$

消除左递归

书 4.3.3

消除左公共因子

书 4.3.3

4.LL（1）表构建

书 4.4.2 和 4.4.3

先算好各个非终结符的 first 和 follow

算法4.31 构造一个预测分析表。

输入：文法G。

输出：预测分析表M。

方法：对于文法G的每个产生式A→α，进行如下处理：

1）对于FIRST（α）中的每个终结符号a，将A→α加入到M［A，a］中。

2）如果∈在FIRST（α）中，那么对于FOLLOW（A）中的每个终结符号b，将A→α加入到M［A，b］中。如果∈在FIRST（α）中，且$在FOLLOW（A）中，也将A→α加入到M［A，$］中。

1. E → TE'　　2. E' → +TE'

3. E' → ε　　4. T → FT'

5. T' → *FT'　　6. T' → ε

7. F → id　　8. F →(E)

First(E)=First(T)=First(F)={(, id}

First(E`)={+, ε}

First(T`)={*, ε}

Follow(E)= Follow(E`)={),$}

Follow(T)= Follow(T`)={+,),$}

Follow(F)={*,+,),$}

	id	+	*	(	)	$
E	E→TE'			E→TE'		
E'		E'→+TE'			E'→ε	E'→ε
T	T→FT'			T→FT'		
T'		T'→ε	T'→*FT'		T'→ε	T'→ε
F	F→ id			F →(E)		

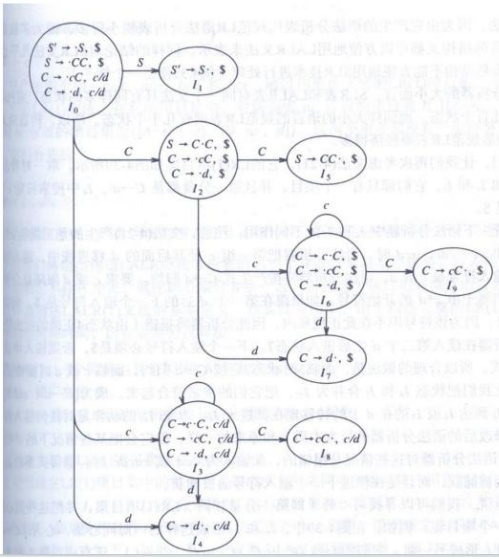
5.LR（1）表构建

书 4.7.2 和 4.7.3

Closure(I)
repeat {
for each item [A→α•Bβ,a] in I
add all [B→• γ ,b] for all b∈FIRST(βa) to I
(if not already in I);
}until (no more items can be added to I);

1. S` →S
2. S →CC
3. C →cC|d

I_0 S′→•S, $
S`→•S,$
S→•CC,$
C→•cC, c|d
C→•d,c|d



state	Action			goto	
	c	d	$	S	C
0	S_3	S_4		1	2
1			acc		
2	S_6	S_7			5
3	S_3	S_4			8
4	r_3	r_3			
5			r_1		
6	S_6	S_7			9
7			r_3		
8	r_2	r_2			
9			r_2		

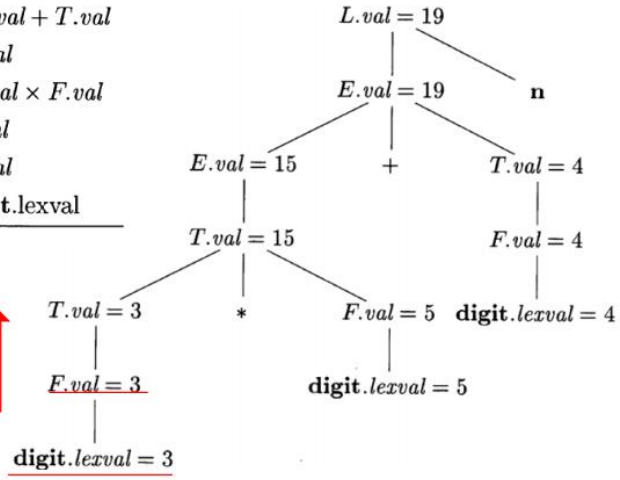
有时间可以再看下书 4.7.5 的 LALR（感觉思路主要是找等价的项集）

6.根据 SDD 构建带注释的语法树

构建语法树后，依据语义规则推理即可。

E.g. Annotated parse tree for 3*5+4 **n**

	产生式	语义规则
1)	$L \to E$ **n**	$L.val = E.val$
2)	$E \to E_1 + T$	$E.val = E_1.val + T.val$
3)	$E \to T$	$E.val = T.val$
4)	$T \to T_1 * F$	$T.val = T_1.val \times F.val$
5)	$T \to F$	$T.val = F.val$
6)	$F \to (E)$	$F.val = E.val$
7)	$F \to$ **digit**	$F.val =$ **digit**.lexval



- 3*5+4n

lexval, val：synthesized attribute

有时间可以再看下书例 5.3，相对来说复杂点，不是从下往上可以直接推（参考图 5-7 的依赖图）

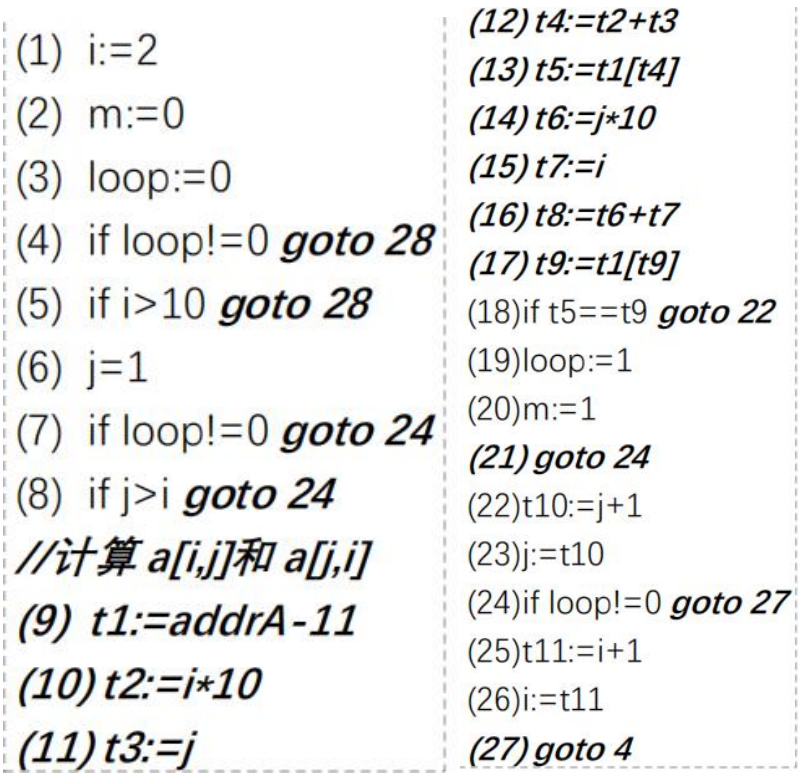
7.C 代码转三地址码

思路上感觉就是逐行翻译，并依次填上 goto 后面的值。注意数组和两个条件与运算这种的翻译。

```
i=2;
m=0;
loop=0;
while (loop==0 && i<=10) {
   j=1;
   while (loop ==0 && j<=i)
      if (a[i,j] != a[j,i]) //"!=" means "not equal to"
      {
         loop=1;
         m=1;

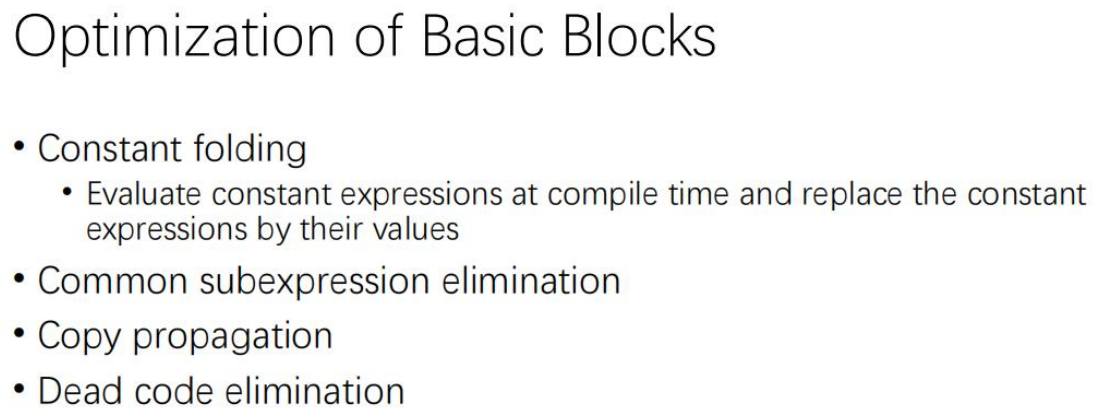

      }
      else j=j+1;
    if (loop==0) i=i+1;
}
```

Notes: Here we assume that the declaration of array A is array [1..10,1..10], each data element of array a would **only use 1 storage unit,** and the start address of array a's storage area is a.



可能有时间去看下短路优化和 backpatching（我也不太会。。。可以看看网课比如哈工大的编译原理 BV1dL4y1H7T8）

8.基本块转 DAG



感觉有关数组操作的 copy propagation 需要注意。以及一些细节，比如优化后序号标记（已经算出值的就不用标序号了）

A source code

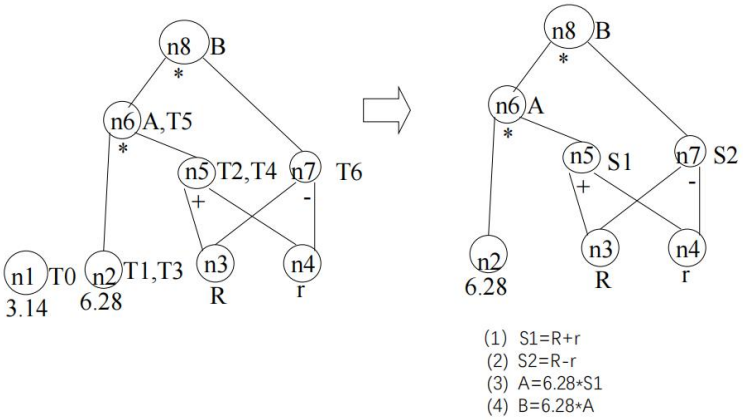
Pi:=3.14

A: = 2*Pi*(R+r);

B:=A;

B:=2*Pi*(R+r)*(R-r)

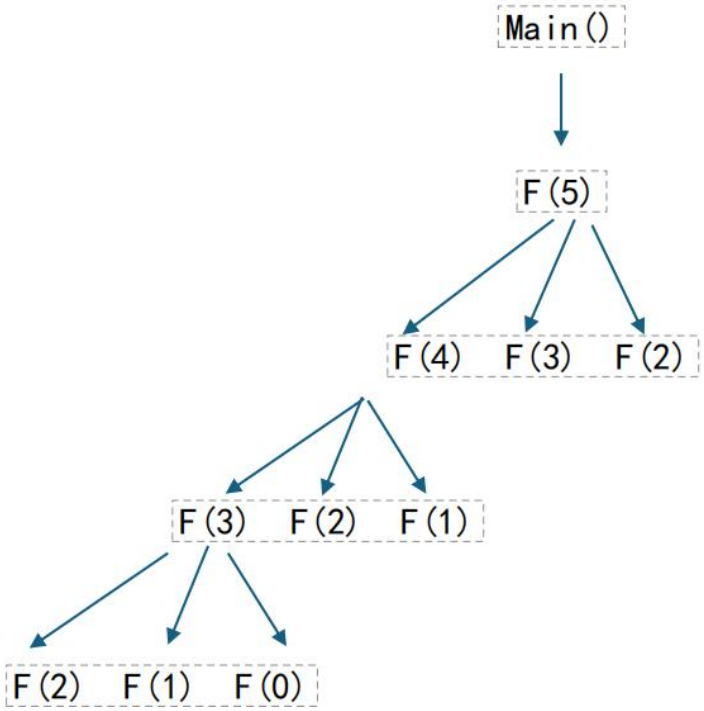
(1)Pi:=3.14
(2)T1:=2*Pi
(3)T2:=R+r
(4)A:=T1*T2
(5)B:=A
(6)T3:=2*Pi
(7)T4:=R+r
(8)T5:=T3*T4
(9)T6:=R-r
(10)B:=T5*T6



(1) S1=R+r
(2) S2=R-r
(3) A=6.28*S1
(4) B=6.28*A

9.C 代码推过程调用时的控制栈内存分配

```
int x,y;
 int main()
 { x=5; y=f(x);}
 int f(int n)
 { if (n<=1)
         return 1;
  else
         if ( n==2) return 2;
        else
         { int t1,t2,t3,t4,t;
           t1=f(n-1); t2=f(n-2); t3=f(n-3); t4=t1+t2;t=t3+t4; return t
         }
 }
```

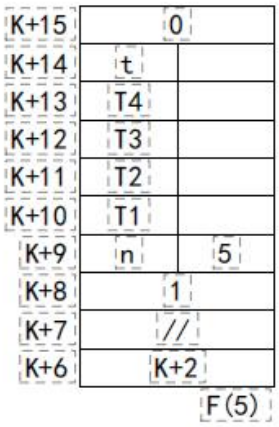


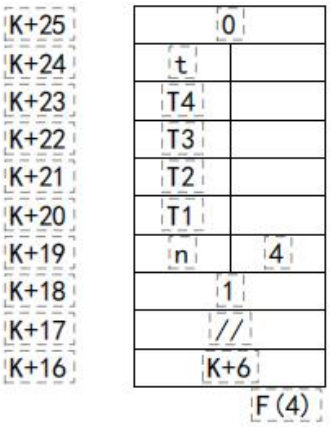
K+1	y	
K	x	5

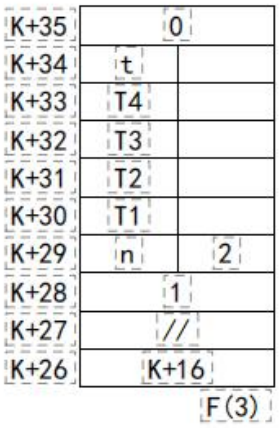
Global variable

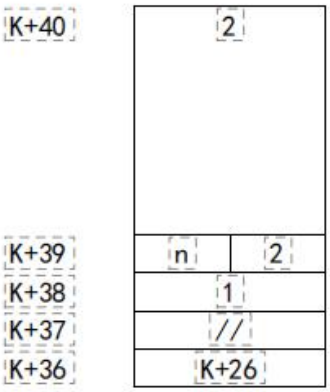
K+5	0
K+4	0
K+3	//
K+2	K

Main()









最后预祝大家都能考出满意的成绩！